

Building Modern Authentication in the Before Time (2015)

└ Who am I?

- Jeffrey Goldberg <https://jeffrey.goldmark.org>
- Of the generation of (dropout) academics who ended up as sysadmins.
- Tried (and failed) to solve the password problem in the 1990s.
- Got into information security and cryptography
- 2010-2023: Worked for 1Password, a password manager.
- Not authorized to speak on behalf of 1Password.

1. Has never taken a programming course, but have taught some.
2. I am much more than that, but trying to keep things relevant and to a single slide.

Building Modern Authentication in the Before Time (2015)

└ Overview

└ 1Password in 2015

- A successful password manager from home users.
- Very strong reputation for privacy as well as security and usability.
- Did not host customer data in any form.
- Hooks for users to sync their data using Dropbox and other third party solutions.

1. E.g., among the first (2012) to encrypt URLs and item titles and to use authenticated encryption (Encrypt-then-MAC).

2026-09-01

Building Modern Authentication in the Before Time (2015)

└ Review

└ Password managers are not based on authentication

└ Penelope and Victor

PENELOPE AND VICTOR

Authentication involves a prover, P , and a verifier, V . V has the power to grant P access to some resource.

- P has access to a secret, s .
- P proves to V that she knows s , typically sending him s .
- V verifies that s is the correct secret for P .
- If the check passes, V grants P access.

1. I'm combining authorization and authentication

2026-09-01

Building Modern Authentication in the Before Time (2015)

└ Review

└ Password managers are not based on authentication

└ Decryption is transformative

DECRYPTION IS TRANSFORMATIVE

- Alice has a small secret, s , perhaps it is a password.
- Alice has two non-secret mathematical functions E and D .
- $c = E(k, m)$, $m = D(k, c)$, and for all m and k :
 $D(k, E(k, m)) = m$

1. s is typically derived from a password.

Building Modern Authentication in the Before Time (2015)

└ Review

└ Cracking

└ Human created passwords are crackable

- Humans can't create high entropy passwords;
- Humans can't remember more than a few medium entropy passwords.



Figure 1: "I'm So Random", xkcd 1230

1. This, of course, is why password managers are necessary, but we will be taking amount the password needed to unlock the password manager.
2. My lame claim to fame is having written a **2011 blog post** (since revised) that inspired the **XKCD "correct horse battery staple" comic**.
3. Joseph Bonneau and Stuart Schechter were able to train people on 56-bit passwords, but it required learning the password incrementally. See

<https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/bonneau>

Building Modern Authentication in the Before Time (2015)

└ Specific authentication concerns

└ Specific concerns

- Server must not receive decryption secrets during authentication;
- Remain secure even if TLS fails;
- Service does not hold onto anything that is crackable.

1. Every password manager with an authentication process must address this problem.
2. TLS was in bad shape at the time. It has dramatically improved since then.
3. The crackability problem is not really about authentication, though it does interact with it. And it would be a separate talk on its own. See “**Secret Key: What is it, and how does it protect you?**” for discussion of that.

Building Modern Authentication in the Before Time (2015)

- └ Specific authentication concerns
 - └ Don't receive decryption secret

In a traditional authentication mechanism, the user password is transmitted to the server, the verifier hashes it, discards the password, and compares the hash to stored hash. Relying on the verification process to discard the secret it receives does not meet the security requirements for a password manager.

Possible solutions

1. Separate password for authentication and decryption
2. Pre-hashing password before transmitting
3. Password-authenticated key exchange (PAKE)

1. Separate authentication password is a tough sell for a product named 1Password.
2. Prehashing alone may make things harder for a malicious server to recover decryption password, but we'd like to do better.
3. The bulk of this talk is about implementing a PAKE in 2015. A PAKE also remains secure if TLS fails.

Building Modern Authentication in the Before Time (2015)

└ General authentication desiderata

└ No secrets transmitted, no replay, no tampering

NO SECRETS TRANSMITTED, NO REPLAY, NO TAMPERING

NS No secrets received
Traditional password authentication emphatically fails to meet this.

NR No replay
An eavesdropper can't learn anything that would allow them to successfully replay an authentication session.

NT No tampering
An active attacker who can tamper with the communication can only prevent successful authentication but cannot learn any secrets.

I will refer to the set of these as "NS+".

1. Previously I erroneously referred to NS as "Zero knowledge". That misuse of the term seems to have caught on. Sorry.

Building Modern Authentication in the Before Time (2015)

└ General authentication desiderata

└ Key Exchange

KE Key exchange

The authentication process results in both sides knowing an encryption key for the session. Proper use of that session key for all communication during the session provides a strong proof that each message comes from a party that successfully went through the authentication process.

1. Proper use means that each message is authenticated (through AEAD or a MAC with a key derived from the session key.)

Building Modern Authentication in the Before Time (2015)

- └ General authentication desiderata

- └ Only the above

There are more slides in this section talking about other desirable properties of an authentication system, but now we will skip ahead to the first slide of the next section, Password-Authenticated Key Exchange (PAKE).

1. I am really tempted to spend time on fixing this cross reference, but I should work on content instead.

Building Modern Authentication in the Before Time (2015)

└ General authentication desiderata

└ Usable

Use: *Reliably and securely usable*

People need to be able to use the system, and the system should be designed to make it easy for people to do things in their own privacy and security interests while hard to do things that go against those interests.

1. This is much harder than one might imagine. For example, most users are not going to distinguish between a password used for authentication versus one for decryption, yet the semantics of a password change for can be catastrophically different.

Building Modern Authentication in the Before Time (2015)

└ General authentication desiderata

└ Undesirable desiderata

UNDESIRABLE DESIDERATA

There are features that may be desirable for systems that may need to avoid or created with strong restrictions for a password manager.

Reset Recoverability

Any entity that has the power to reset a forgotten or lost authentication secret has the capability to to take over the account.

Alt Alternative authentication

Sometimes what is labeled as two-factor authentication is better described as alternative factor authentication. This is extremely dangerous if users are led to believe that it helps keep attackers out.

1. Oxymoronic slide title.
2. 1Password's recovery mechanism is nightmarishly complicated, and would be a topic for another day.

Building Modern Authentication in the Before Time (2015)

- └ General authentication desiderata

- └ Distinct factors

MF Multiple factor

If at least one of multiple authentication factors is not compromised, the authentication process retains its security properties.

1. This is widely and often dangerously misunderstood. And is a separate topic.

Building Modern Authentication in the Before Time (2015)

└ Password-Authenticated Key Exchange (PAKE)

└ Enrollment

ENROLLMENT

1. Generate long term client secret, x , from password, salt (s), etc.
2. Compute a mathematically related verifier, v .
3. Send v and s to service.

1. For the PAKE we ended up using the long term client secret is called x , so we are stuck with that.
2. I am describing an augmented PAKE, in which it is infeasible to compute x from v .
3. Sending v to the service is the only time a secret is transmitted

Building Modern Authentication in the Before Time (2015)

└ Password-Authenticated Key Exchange (PAKE)

└ Authentication: Compute shared K

1. Client generates an ephemeral client secret, a .
 2. Client generates an ephemeral public key, A , from x and a .
 3. Client sends A (along with identity, I) to server.
 4. Server generates an ephemeral server secret, b .
 5. Server generates an ephemeral public key, B , from v and b .
 6. Server sends B to client.
 7. Client computes K_c from x , a , and B .
 8. Server computes K_s from v , b , and A .
- If x and v have the right mathematical relation, then $K_c = K_s$.

1. Identity is something like username.
2. The computations, sequence of computation, nature of public parameters used, and even the number of messages back and forth depend on the particular PAKE.

Building Modern Authentication in the Before Time (2015)

└ Password-Authenticated Key Exchange (PAKE)

└ Authentication: Check shared K

1. Details don't matter (and there are different ways to do this.) What matters is that there is a way for the client and the server to each prove to each other that they have the same K without revealing any secrets.

$K_x = K_y$ only if x and y have the right mathematical relation.

Notation

- " $\oplus$ " is exclusive or; " $\parallel$ " is reversible concatenation;
- H , an agreed upon cryptographic hash function;
- g , a public parameter used computations;
- s , the salt, shared during enrollment;
- I , the client's identity, set during enrollment;
- K_c and K_s , keys that the client (C) and server (S) have computed.

1. C sends $M_1 = H(N) \oplus H(g) \parallel H(I) \parallel s \parallel A \parallel B \parallel K_c$ to S.
2. S verifies M_1 .
3. S sends $M_2 = H(A \parallel M_1 \parallel K_s)$ to C.
4. C verifies M_2 .

Building Modern Authentication in the Before Time (2015)

└ Password-Authenticated Key Exchange (PAKE)

└ Choosing a PAKE: Criteria

- Existing libraries we could use;
- Augmented PAKE ($x \neq v$)
- Clear of any patents.

1. The reason for seeking an augmented PAKE is so capture of the server stored verifier v could not lead to impersonation of a client.
2. At the time were were some contestable patents on categories of PAKEs, and I lived in the Eastern Federal District of Texas.

Building Modern Authentication in the Before Time (2015)

- └ Password-Authenticated Key Exchange (PAKE)

- └ Leaks like a sieve



Figure 2: Leaks like a sieve

1. tbh, neither a spatter screen nor a colander are sieves, but they are what I had at home.

Building Modern Authentication in the Before Time (2015)

└ Password-Authenticated Key Exchange (PAKE)

└ A once fresh meme

Do not use SPR for any new projects today. It was a necessary choice for 1Password in 2015; there are far better PAKEs available now.



Figure 3: Something I tweeted a year after launch

1. OPAQE is also passé. Consult your nearest cryptographer for current PAKE recommendations.

Building Modern Authentication in the Before Time (2015)

└ Lessons from rolling our own

└ Developers need guidance

Pointing even very talented developers at a standard and expecting them to implement well if they don't understand why the protocol works leads to problems.

- Without an understanding of how the pieces fit together, code can be wildly misfactors.
- Developers didn't know which parts of computation where cryptographic secrets.
- Allowed a seemingly harmless shortcut in not padding a number before hashing that lead to a Heisenbug years later.

1. I failed to provide enough guidance during development.
2. Poor code organization contributed to my failure to notice that an important check was not being performed. Bounty hunter found during testing that check that client ephemeral public, A was not being checked for $A \not\equiv 0 \pmod{N}$.
3. Most anyone who has had some course in cryptography will be familiar with the convention during key exchange that little a is an ephemeral client secret and big A is an ephemeral public key. But most developers won't know that. Only after seeing little a not properly handled did I change our variable name to `ephemeralClientSecret`.